University of Bergen - Department of Informatics

Top Trading Cycles in a practical setting

Author: Lars Møen Haukland

Supervisor: Fredrik Manne

UNIVERSITETET I BERGEN
Fakultet for naturvitenskap og teknologi

June, 2026

Abstract

This thesis explores the Top Trading Cycles algorithm in the practical setting of the allocations of GPs, General Practitioner, in Norway for people wanting to switch or get a GP. It explores how different focus of *good* results can change both the runtime and results, and how to make the algorithm run as efficiently as possible. This thesis proposes an implementation of the Top Trading Cycles that can reduce the waitlists for GPs in norway by over 80%.

Acknowledgements

I would like to thank my supervisor Fredrik Manne for his guidance and support throughout this project. I would also like to thank Juni Weisteen Bjerde for input and help during the project. In addition Juni and Mathias Bertntert together theorised the exact algorithms so that I could implement them.

Contents

1. Introduction

The Norwegian general practitioner (GP) system, started in 2001, has aim to serve the Norwegian population with a stable GP. A GP can follow his or hers patients over time, creating a trustful and safe relationship [1]. Today this system works as intended, and GPs handle most of the populations needs. As of December 2025 the number of GPs in Norway is 5720 and the total number of citizents in the Norwegian GP system is 5.6 million. The amount of patients each GP has varies but on average each GP has about a thousand patients [2].

One inefficiency in the GP system is when a person wants to switch from one GP to another. If the GP they want to switch to does not have any more capacity for new patients, the person has to sign up in a queue and wait for a free slot to open up with that GP. In December the number of open GP lists was 1340, indicating that 77% of all GPs had no extra capacity [3]. In a more densely populated city such as Bergen, the numbers are even worse. In December 2025 Bergen had 268 GPs and only 7 GPs that had available capacity for new patients. Thats 97% of the GPs in Bergen had no extra capacity [3].

When a person wants to switch to a GP that has no excess capacity they must sign up and then wait in line. In December 2025 there were about 300,000 people waiting for GPs in Norway [2]. As there are only 5720 doctors each doctor has on average more than 50 people on his or hers waiting list. Since many are switching from a GP it follows that it is likely that there are cycles, in that a person P_A wants doctor D_A which currently has P_B as a patient, but P_B wants to switch to doctor D_B who currently has P_A as a patient. In this case patient P_A ends up waiting for an open slot with P_B's doctor and vice versa, but logically instead of waiting P_A and P_B could just swap doctors. This can be generalized to longer "waiting cycles" that could all be serviced by allowing for simultaneous switching.

In this thesis, we study this problem and come up with solutions for how one can help people who are waiting in line so that they can get their preferred choice. In particular we propose that we can use cycle detecting algorithms, to find optimal switches between patients and massively reduce the number of people on waiting lists by using these algorithms periodically. Our algorithms are based on existing cycle cancelling and TTC algorithms, and focus on two measures of fairness:

- Switching as many patients as possible
- Switching as high priority patients as possible

We have developed and tested algorithms for these cases.

In addition we compare these optimal algorithms with the existing TTC algorithm and a greedy algorithm, comparing gain of good vs runtime.

1.1. Thesis Outline

The remainder of this thesis is organized as follows:

- **Section 2** provides background on the Top Trading Cycles mechanism, cycle cancelling algorithms and how these methods have been used in similar problems.
- **Section 3** formally defines the computational problems arising from TTC as variants of cycle cover problems on directed graphs.

- **Section 4** describes our Rust implementation and the algorithmic strategies employed.
- **Section 5** presents experimental results comparing different priority strategies.
- **Section 6** concludes and discusses future work.

2. Background

Problems like the GP allocation problem have been studied for quite some time with a lot of variations.

2.1. Similar problems

2.2. Top Trading Cycles

2.3. Related Work

3. Problem Formalization

In this chapter we define the GP allocation problem and also some variations of it. In addition we go through how to construct a graph given a GP allocation problem and finally how we can reduce the GP allocation problem to the Cycle Cover problem in directed graphs.

3.1. The GP allocation problem

3.1.1. Input

In the GP allocation problem we are given a set of patients $P = \{p_1, p_2, ..., p_n\}$ and a set of doctors $D = \{d_1, d_2, ..., d_m\}$. We are also given the current and preferred GP assignments, for each patient:

- $D_{\text{cur}}[i]$ denotes the index of the current doctor assigned to patient p_i (or $\perp$ if unassigned)
- $D_{\text{pref}}[i]$ denotes the index of the preferred doctor for patient p_i

In addition we are given a priority function $R : P \to \mathbb{N}$, mapping some positive integer to each patient, with higher numbers indicating a higher priority to help this patient.

3.1.2. Feasible solution

A feasible solution for the GP allocation problem is a subset of patients $S \subseteq P$ such that the directed graph

$$G_S = (S, E_S), E_S = \{(a, b) \mid a, b \in S, D_{\text{pref}}[\text{idx}(a)] = D_{\text{cur}}[\text{idx}(b)]\} \quad (1)$$

consists of a vertex-disjoint union of directed cycles that cover all vertices in S. This means that S is a set of patients that can all get their preferred doctor if we allow for simultanous exchanges following cycles.

3.1.3. Optimization criterion

Next we can start defining variants of feasible solutions where we want to maximize some *score*. Examples of such a scoore could be to find a feasible solution with many patients or something based on priority. We might want a solution that exchanges as many patients as possible, exchanges patients with the highest priority or some mix of these.

First we define our general ordering

> **Definition 3.1.3.1** (Optimization criterion): An ordering $\succ$ over feasible solutions. A solution is optimal if it is maximal under $\succ$.

Build on this to create our three main variants of scoring functions.

3.1.3.1. Lexicographic maximization by priority

Definition 3.1.3.1.1 (Characteristic vector): Let patients be ordered by priority: $p_1 \geq p_2 \geq \ldots \geq p_n$ where p_1 has the highest priority. Given a feasible solution $S \subseteq P$, the **characteristic vector** is:

$$\chi(S) = (b_1, b_2, \ldots, b_n) \in \{0, 1\}^n, \quad b_i = \begin{cases} 1 \text{ if } p_i \in S \\ 0 \text{ otherwise} \end{cases} \tag{2}$$

Definition 3.1.3.1.2 (Lexicographic ordering by priority):

$$S \underset{\text{lex}}{\succ} S' \quad \text{iff} \quad \chi(S) \text{ is lexicographically greater than } \chi(S') \tag{3}$$

That is, at the first index i where S and S' differ, $\chi(S)_i = 1$ and $\chi(S')_i = 0$.

Intuitively, $\chi(S)$ is a binary number whose most significant non-zero bit corresponds to the highest-priority patient that gets helped. The optimal solution is then the one that maximises this binary number: always satisfying the highest-priority patient it can, then subject to that the next highest, and so on.

Example (Ordering two solutions lexicographically by priority):

We use the example graph in Figure 1 as the graph to create solutions from.

Given solutions
1. $S = \{P_4, P_5\}$ represents the subgraph G_S containing the cycle $P_4 \to P_5 \to P_4$
2. $S' = \{P_1, P_2, P_3\}$ represents the subgraph $G_{S'}$ containing the cycle $P_1 \to P_2 \to P_3 \to P_1$

For simplicity's sake we say that $R(P_i) = i$, so P_5 has the highest priority.

Ordering all five patients by priority gives $p^{(1)} = P_5, p^{(2)} = P_4, \ldots, p^{(5)} = P_1$. Then:

$$\chi(S) = (1, 1, 0, 0, 0) \quad \chi(S') = (0, 0, 1, 1, 1) \tag{4}$$

At the first differing position ($i = 1$), $\chi(S)_1 = 1 > 0 = \chi(S')_1$, so $S \underset{\text{lex}}{\succ} S'$.

So for our first variant we want to find the maximal solution under the $\underset{\text{lex}}{\succ}$ ordering. This means always prioritizing patients with highest priority.

3.1.3.2. Maximizing cardinality

Another natural variant is to find a solution with the largest cardinality, e.g. a solution that contains the most patients.

Definition 3.1.3.2.1 (Ordering by cardinality):

$$S \underset{\text{size}}{\succ} S' \text{ iff } |S| > |S'| \tag{5}$$

This optimal solution will then be one that exchanges the most patients.

Example (Ordering two solutions by cardinality):

Using the same solutions as in the previous example:

1. $S = \{P_4, P_5\}$ represents the subgraph G_S containing the cycle $P_4 \to P_5 \to P_4$
2. $S' = \{P_1, P_2, P_3\}$ represents the subgraph $G_{\{S'\}}$ containing the cycle $P_1 \to P_2 \to P_3 \to P_1$

Under $\underset{\text{size}}{\succ}: |S| = 2$ and $|S'| = 3$, so $S' \underset{\text{size}}{\succ} S$. However the solution is $\{P_1, P_2, P_3, P_4, P_5\}$, the union of both cycles.

Notice that the same two sets are reversibly ordered under the two criteria: $S \underset{\text{lex}}{\succ} S'$ (because S contains the highest-priority patient P_5) but $S' \underset{\text{size}}{\succ} S$ (because S' has more patients).

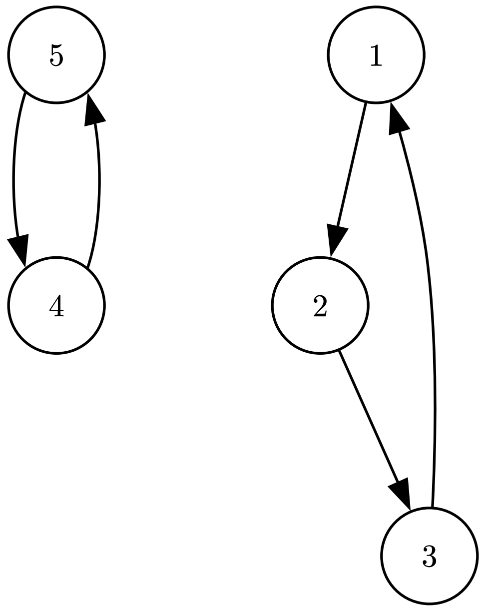

Figure 1: An example graph G.

3.1.3.3. Maximizing utility

Finally we can formulate an ordering that is somewhat of a combination of $\underset{\text{lex}}{\succ}$ and $\underset{\text{size}}{\succ}$ which is the total utility of a solution. We define the total utility to be sum of priorities.

Definition 3.1.3.3.1 (Total utility function):

$$U(S) = \sum_{p \in S} R(p) \tag{6}$$

Definition 3.1.3.3.2 (Ordering by total utility):

$$S \underset{\text{util}}{\succ} S' \text{ iff } U(S) > U(S') \tag{7}$$

This means that even though solution S' contains some high priority patients, if S contains enough lower priority patients then S can be more maximal under $\underset{\text{util}}{\succ}$.

Example (Ordering two solutions by total utility):

We use the following solutions from Figure 2:
1. $S = \{1, 3, 4\}$ representing the cycle $P_1 \to P_3 \to P_4 \to P_1$
2. $S' = \{5, 2\}$ representing the cycle $P_5 \to P_2 \to P_5$

Now the total utility is as follows:
$U(S) = 1 + 3 + 4 = 8$
$U(S') = 2 + 5 = 7$

So $S \underset{\text{util}}{\succ} S'$, here we see that its the combination of more patients with lesser priority that can make a solution be better under $\underset{\text{util}}{\succ}$ than another with greater priority.

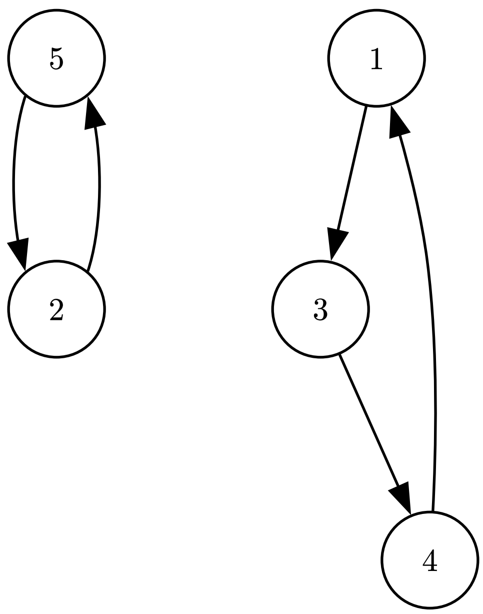

Figure 2: An example graph G.

3.1.4. Priority function

The priority function has quite a large effect on the switches we end up taking. As the priority of a patient decides if that patient will be in the solution or not for most of the algorithms, except for the exact algorithm maximizing cardinality $\underset{\text{size}}{\succ}$. This is why its important to discuss what effects the function can have on solutions when we are maximizing the ordering $\underset{\text{util}}{\succ}$.

If we make the priority function exponential such that $R(a) = 2^a$ then using the ordering $\underset{\text{util}}{\succ}$ becomes equal to $\underset{\text{lex}}{\succ}$. While this ordering is in terms maybe the most fair it might not always be the best for the collective good, since a high priority patient might block a lot of lower priority patients from switching. This way we might want a priority function like $R(a) =$ Days patient a has been waiting for a switch. This way if we have the choice between patient P_i who has waited for 30 days $R(P_i) = 30$ and 4 patients who have waited 40 days in total an algorithm maximizing $\underset{\text{util}}{\succ}$ will choose the 4 patients.

We can adjust the priority function to prioritize higher priority by making it exponential but still under the 2^a. When we use days waited we can make $R(a) = 1.1^{\text{Days patient a has been waiting}}$ Adjusting this closer to 2 makes higher priority patients more prioritized and vice versa for lowering it.

4. Implementation

In this chapter we look at each algorithm we have implemented, how we implemented them, why and runtime analysis. We have in total implemented 5 algorithms, TTC, Greedy DFS, Exact Cardinality, Exact Priority and a specialized case of Exact Priority. For the exact algorithms we also give, or refer to existing, proofs for why they are exact.

4.1. Different graph representations used

In the algorithms we use different graph representations, so we start by defining the different representations of *The GP allocation problem.*

4.1.1. Patient and Doctor graph

First we have the easiest graph containing both patients and doctors as vertices. The edges are directed from a patient to a doctor and from a doctor to a patient. Edges never go patient $\to$ patient or doctor $\to$ doctor. An edge patient $\to$ doctor symbolizes that the patient has that doctor as its preferred doctor, and doctor $\to$ patient symbolises that the doctor currently has that patient as one of his or hers current patients. Observe that this graph is bipartite as we have patients on one side and doctors on the other. Now the formal definition of our graph. Let $I = \{0, ..., |P| - 1\}$. Then:

$$G = (V, E), \quad V = P \cup D, \quad E = \left\{ \left(p_i, D_{\text{pref}[i]}\right) \mid i \in I \right\} \cup \left\{ \left(D_{\text{cur}[i]}, p_i\right) \mid i \in I \right\} \tag{8}$$

4.1.2. Doctor graph collapsed edges

In this weighted graph we condense the problem to only have doctors as nodes and edges between doctors. An edge doctor a $\to$ doctor b symbolises that there exists a patient that wants to switch from doctor a to doctor b, or that the patient currently has doctor a and has doctor b as preferred. We define our graph as:

$$\begin{gathered} G = (V, E), \quad V = D \\ E = \left\{ \left(D_{\text{cur}[i]}, D_{\text{pref}[i]}\right) \mid i \in I \right\} \\ w(a, b) = \left|\left\{ i \in I \mid D_{\text{cur}[i]} = a \land D_{\text{pref}[i]} = b \right\}\right| \end{gathered} \tag{9}$$

4.2. Greedy DFS

We first consider a greedy approach inspired by the Top Trading Cycles implementation of Huitfeld et al. Their algorithm preserves TTC properties at each round by restricting each doctor node to a single outgoing edge. We relaxed this constraint, allowing each doctor to maintain outgoing edges to all of its current patients simultaneously, and resolved ties by patient priority.

We are given the lists of patients and doctors P, D, the assignment arrays $D_{\text{cur}}, D_{\text{pref}}$, and a priority function $R : P \to \mathbb{N}$, where a higher number means higher priority. We model the problem as a bipartite directed graph over patients and doctors. Let $I = \{0, ..., |P| - 1\}$. Then:

$$G = (V, E), \quad V = P \cup D, \quad E = \left\{ \left(p_i, D_{\text{pref}[i]}\right) \mid i \in I \right\} \cup \left\{ \left(D_{\text{cur}[i]}, p_i\right) \mid i \in I \right\}$$

Each patient p_i has an outgoing edge to their preferred doctor, and each doctor has outgoing edges to all of its currently registered patients. A directed cycle in G necessarily alternates between patient and doctor nodes and corresponds to a valid exchange: each patient in the cycle moves to their preferred doctor, and each doctor loses exactly one patient while gaining one.

The algorithm processes patients in decreasing priority order. For each patient, a DFS attempts to find a cycle through that patient; when the DFS reaches a doctor node with multiple candidate patients, it always explores the highest-priority one first.

```
let resolved_patients = []
let p_prio = Sorted list of patients by priority
let wants_to_switch = [true] ∗ |P|
for each p ∈ p_prio do
  if let cycle = dfs_find_cycle(G, R, p) then
    for each p ∈ cycle do
      let i = idx(p)
      wants_to_switch[i] = false
      resolved_patients.push(p)
    end
  end
end
return resolved_patients
```

The greedy rule ensures that high-priority patients are preferentially included in cycles, but it does not guarantee a globally optimal solution. The following example illustrates how the greedy choice can be locally motivated yet globally suboptimal.

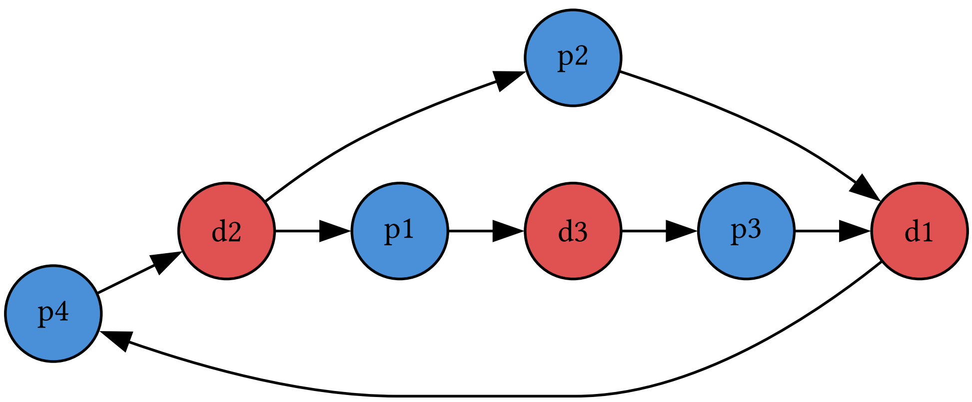

Figure 3: Example graph G where algorithm would choose suboptimal solution. px means patient x and has priority x.

In Figure 3 each patient px has priority x. The DFS begins at $p4$ (highest priority), follows the edge to $d2$, and there chooses $p2$ over $p1$ since $R(p2) > R(p1)$. The resulting cycle $p4 \to d2 \to p2 \to d1 \to p4$ is committed, leaving $p1$ and $p3$ unsatisfied with no further cycles remaining.

Had the DFS chosen $p1$ at $d2$ instead, it would have found the longer cycle $p4 \to d2 \to p1 \to d3 \to p3 \to d1 \to p4$, satisfying three patients. The greedy choice at $d2$ was locally motivated by priority but globally suboptimal. This motivates the exact algorithms in the following sections.

4.3. Exact algorithm for maximizing total switches

NB: HER OG NESTE ER DET MYE AI, NOE JEG VIL HØRE DERES MENING OM. NOE AV DETTE SYNES JEG ER SKREVET BRA, ANDRE LITT USIKKER. INKLUDERT KILDENE ER IKKE RIKTIG

This algorithm applies the classical cycle canceling technique from network flow theory to the GP-switching problem, finding the maximum-cardinality feasible solution in polynomial time [4, §9.6]. The key insight is that the problem reduces to a maximum integer circulation, for which efficient algorithms are known.

4.3.1. Graph structure

Rather than working with the bipartite patient-doctor graph, we compress to a weighted directed graph over doctors only. Patients who want the same switch are interchangeable: all that matters is how many can be routed. We therefore aggregate them into edge weights:

$$\begin{gathered} G = (V, E), \quad V = D \\ E = \left\{ \left(D_{\mathrm{cur}[i]}, D_{\mathrm{pref}[i]} \right) \mid i \in [\![P]\!] \right\} \\ w(a, b) = \left| \left\{ i \in [\![P]\!] \mid D_{\mathrm{cur}[i]} = a \land D_{\mathrm{pref}[i]} = b \right\} \right| \end{gathered} \tag{10}$$

A cycle $d_1 \to d_2 \to \cdots \to d_k \to d_1$ carrying f units of flow corresponds to f patients rotating around the cycle: each moves from their current doctor to the next in the cycle. This compression reduces the graph from $O(|P| + |D|)$ nodes to $O(|D|)$ nodes, which is significant when many patients share the same switch request.

Applying this transformation to Figure 3 gives the doctor graph in Figure 4, where we can still identify the short cycle $d1 \to d2$ and the longer cycle $d1 \to d2 \to d3$.

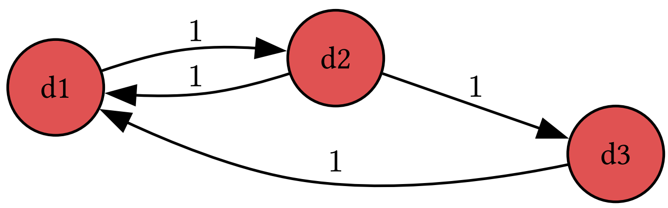

Figure 4: Graph from Figure 3 but in doctor graph structure.

4.3.2. Algorithm

We want to find, for each edge $e \in E$, a non-negative integer $f(e)$ representing how many of the $w(e)$ patients on that edge actually switch. Two conditions make such an assignment a valid set of simultaneous exchanges:

- **Capacity:** $0 \le f(e) \le w(e)$ for all $e \in E$

- **Flow conservation:** for every doctor $d \in D$: $\sum_{(v,d)\in E} f(v,d) = \sum_{(d,v)\in E} f(d,v)$

Flow conservation is exactly what forces the assignment to decompose into cycles: every doctor who loses a patient must gain one. The problem is therefore:

> **Problem: Maximum Switch Circulation**
>
> **Input:** A directed graph $G = (D, E, w)$ where $w : E \to \mathbb{N}$
> **Output:** An integer function $f : E \to \mathbb{N}_0$ maximising $\sum_{e\in E} f(e)$ subject to:
>
> - $0 \leq f(e) \leq w(e)$ for all $e \in E$
> - $\sum_{(v,u)\in E} f(v,u) = \sum_{(u,v)\in E} f(u,v)$ for all $u \in D$

This is a **maximum integer circulation** problem, solvable in polynomial time. We solve it using the **successive positive cycle** method.

Given a current flow f, the **residual graph** G_f is built by replacing each original edge (u, v) with:

- a **forward residual** arc (u, v) with capacity $w(u, v) - f(u, v)$ and cost $+1$
- a **backward residual** arc (v, u) with capacity $f(u, v)$ and cost -1

A directed cycle in G_f is **positive** if the sum of its arc costs is positive, meaning it contains more forward than backward arcs, so augmenting along it strictly increases total flow. The optimality condition for maximum circulations states that f is optimal if and only if G_f contains no positive cycle.

The outer loop finds and augments positive cycles until none remain:

for each $e \in E$ **do** $f(e) \leftarrow 0$ **end**
while FindPositiveCycle(G_f) returns a cycle C **do**
  $b \leftarrow \min_{e\in C} r_f(e)$ ▷ bottleneck residual capacity
  for each $(u, v) \in C$ **do**
    $f(u, v) \leftarrow f(u, v) + b$; update residual capacities in G_f
  end
end
return f

Positive cycles are detected using a variant of Bellman–Ford (SPFA). All nodes are seeded with distance 0, simulating a virtual super-source at zero cost so every cycle is reachable. Edges are relaxed greedily in the maximising direction; a node relaxed $|D|$ or more times must lie on a positive cycle.

for each $v \in D$ **do** dist$[v] \leftarrow 0$; pred$[v] \leftarrow \bot$; enqueue v **end**
while queue not empty **do**
  $u \leftarrow$ dequeue

for each (u, v) in G_f with $r_f(u, v) > 0$ **do**
  if $\text{dist}[u] + 1 > \text{dist}[v]$ **then**
    $\text{dist}[v] \leftarrow \text{dist}[u] + 1$; $\text{pred}[v] \leftarrow u$
    if v has been enqueued $\geq |D|$ times **then**
      Walk back $|D|$ steps from v along pred to find node c on the cycle
      Collect and **return** the cycle starting and ending at c
    end
    Enqueue v if not already in queue
  end
end
end
return $\emptyset$ ▷ no positive cycle; f is optimal

Theorem 4.3.2.1 (Optimality of MaxSwitchCirculation): The algorithm returns a flow f achieving the maximum possible value $\sum_{e \in E} f(e)$, equal to the maximum number of patients that can simultaneously switch to their preferred doctor.

Theorem 7: Optimality of MaxSwitchCirculation

Proof: The algorithm terminates when FindPositiveCycle returns $\emptyset$, meaning G_f contains no positive-cost directed cycle. By the cycle optimality criterion for maximum circulations: a feasible integer circulation f is optimal if and only if its residual graph G_f contains no positive-cost cycle. Since each forward arc carries cost $+1$ and each backward arc cost -1, any positive-cost cycle in G_f would strictly increase $\sum f(e)$; the absence of such cycles certifies that f is maximum. □

Termination and complexity. Each call to FindPositiveCycle runs SPFA over $|D|$ nodes and at most $|D|^2$ edges in $O(|D|^3)$ time. Each augmentation round increases total flow by at least 1, so the number of rounds is at most $W = \sum_{e \in E} w(e)$, the total number of patients wanting to switch, giving a worst-case bound of $O(W \cdot |D|^3)$. In practice each round pushes a bottleneck of $b \geq 1$ units, so the number of distinct rounds is bounded by the number of edges that become saturated, at most $|E| \leq |D|^2$. This gives the graph-size bound $O(|D|^5)$, independent of the patient count.

4.4. Exact algorithm for maximizing priority

This algorithm finds the lexicographically maximal feasible solution under $\underset{\text{lex}}{\succ}$, as defined in Section 3. It builds on the same residual-graph framework but processes patients in priority order, greedily committing each one as soon as a cycle is found. The correctness of this greedy strategy follows from the theorem below.

Theorem 4.4.1 (Greedy choice property): Let p^* be the highest-priority patient that belongs to some feasible solution. Then every lexicographically maximal solution contains p^*.

Theorem 8: Greedy choice property

Proof: Suppose S^* is a lexicographically maximal feasible solution with $p^* \notin S^*$. By hypothesis there exists a feasible solution S' with $p^* \in S'$. Let k be the index of p^* in the priority ordering $p^{(1)} \succ p^{(2)} \succ \cdots \succ p^{(n)}$. Every patient with index $i < k$ belongs to no feasible solution by the choice of p^*, so $\chi(S^*)_i = \chi(S')_i = 0$ for all $i < k$. At index k: $\chi(S')_k = 1 > 0 = \chi(S^*)_k$. Therefore $S' \underset{\text{lex}}{\succ} S^*$, contradicting the maximality of S^*. □

The algorithm applies this observation iteratively: process patients from highest to lowest priority and commit each patient to a cycle as soon as one is found.

We use the same doctor graph as before: doctors are nodes, and each patient p with current doctor u and preferred doctor v is a directed arc $a_p = u \to v$ with capacity 1. Each arc also has a corresponding **backward arc** $\overline{a_p} = v \to u$ with initial capacity 0.

Pruning. Before processing, we compute the strongly connected components (SCCs) of the graph. Any patient whose current and preferred doctor lie in different SCCs, or in a trivial SCC of size 1, can never be part of any directed cycle and is immediately discarded. This avoids unnecessary BFS calls and keeps the residual graph clean throughout execution.

The algorithm processes the remaining patients from highest to lowest priority. For each patient p (arc $u \to v$), exactly one of two cases applies:

- **Case 1 (primary arc available,** $\operatorname{cap}(a_p) > 0$**):** p has not yet been committed. We run BFS from v to u in the current residual graph. If a path Π exists, then Π together with a_p forms a cycle, and we commit p:
  - The primary arc a_p is consumed **permanently**: its backward arc capacity stays 0, so no future patient can traverse it in reverse and undo p's switch.
  - Each routing arc a in Π is consumed normally: $\operatorname{cap}(a)$ decreases by 1 and $\operatorname{cap}(\overline{a})$ increases by 1, leaving a residual that lower-priority patients may use to reroute.
  - If no path exists, p is skipped.
- **Case 2 (primary arc consumed, residual active,** $\operatorname{cap}(a_p) = 0$ and $\operatorname{cap}(\overline{a_p}) > 0$**):** p's arc was already consumed as a **routing arc** in an earlier (higher-priority) patient's cycle, which created the residual $\overline{a_p}$. We now **solidify** by setting $\operatorname{cap}(\overline{a_p}) \leftarrow 0$, preventing any lower-priority patient from traversing this residual to undo p's assignment.

Compute SCCs of G; discard patients whose arc spans different or trivial SCCs
$E_{\text{sort}} \leftarrow$ remaining patients sorted by priority descending
for each patient p with arc $a_p = (u \to v)$ in E_{sort} **do**
  if $\operatorname{cap}(a_p) > 0$ **then**
    if $\operatorname{BfsPath}(v, u)$ in residual graph returns path Π **then**

$\text{cap}(a_p) \leftarrow 0$ ▷ consume primary permanently; no residual
for each arc $a \in \Pi$ **do**
$\text{cap}(a) \leftarrow \text{cap}(a) - 1$; $\text{cap}(\overline{a}) \leftarrow \text{cap}(\overline{a}) + 1$
end
Add p to S
end
else if $\text{cap}(\overline{a_p}) > 0$ **then**
$\text{cap}(\overline{a_p}) \leftarrow 0$ ▷ solidify; lock in routing commitment
Add p to S
end
end
return S

Why the primary arc has no residual: This is the core invariant. Once patient p is committed as the beneficiary of a cycle, no future patient should be able to undo it. By not creating a residual for a_p, no BFS can ever traverse backwards through p's switch.

Why routing arcs do have residuals: A lower-priority patient may be able to "take over" a routing role. For example, if p_1 (high priority) uses p_3's arc as routing, a later patient p_2 may form a cycle that routes through p_3's arc differently, replacing it. This is valid: only the final destination of p_1 (whose primary arc is irrevocable) is fixed.

Theorem 4.4.2 (Correctness): The algorithm returns the lexicographically maximal feasible solution.

Theorem 9: Correctness

Proof: By induction on the priority ordering. The Greedy Choice Theorem establishes the base case: the highest-priority feasible patient p^* must appear in every lex-max solution, and BFS correctly identifies whether p^* is feasible, since a path from v to u exists in G if and only if p^* lies on some directed cycle.

For the inductive step, assume all higher-priority patients have been correctly committed. The residual graph reflects these commitments: primary arcs are permanently consumed and cannot be undone, while routing residuals remain available. We claim a path from v to u exists in the current residual graph if and only if p can be added to a feasible solution extending all committed patients. This follows from the augmenting-path argument: any feasible cycle through p either avoids all committed arcs and is directly reachable in G_f, or shares some routing arcs whose residuals permit an equivalent rerouting. The solidify step in Case 2 maintains the invariant: once a patient's arc is confirmed as routing for a higher-priority cycle, its residual is removed so no lower-priority patient can undo the commitment. □

Complexity. Sorting patients requires $O(|P| \log|P|)$. The initial SCC computation runs in $O(|D| + |E|)$. Each patient requires at most one BFS over the residual graph in $O(|D| + |E|)$ time. Processing all $|P|$ patients therefore takes $O(|P| \cdot (|D| + |E|))$. Since $|E| \leq |D|^2$, the overall complexity is $O(|P| \cdot |D|^2)$.

4.5. Metaheuristics

5. Experimental Results

5.1. Experimental Setup

5.2. Performance Comparison

5.3. Impact of Priority Strategies

6. Conclusion

6.1. Summary

6.2. Future Work

Bibliography

[1] Legelisten, “Med rett til å bytte – Ventetider for å bytte fastlege i Norge.” Accessed: Apr. 08, 2026. [Online]. Available: https://legelisten.s3.eu-west-1.amazonaws.com/blog-files/Med+rett+til+%C3%A5+bytte+-+Ventetider+for+%C3%A5+bytte+fastlege+i+Norge.pdf

[2] Helfo, “Fastlegestatistikk.” Accessed: Apr. 08, 2026. [Online]. Available: https://www.helfo.no/Fastlegeordninga/fastlegestatistikk

[3] Statistisk sentralbyrå, “Fastlegelister og fastlegekonsultasjoner, etter år og region (12005).” Accessed: Apr. 08, 2026. [Online]. Available: https://www.ssb.no/statbank/table/12005

[4] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications.* Prentice Hall, 1993.

Index of Figures